ISW - Tutorato

s.dessi63@studenti.unica.it

l.piras78@studenti.unica.it

OGGETTO MAIL “ISW”

Cosa vedremo oggi

- Stringhe
- Liste
- Slicing
- Try & except
- Esercizi, esercizi, esercizi !

Stringhe

Python – Tipi di dato

Tipo di dato	Nome	Descrizione	Esempi
Intero	int	Intero di dimensione arbitraria	-42, 0, 1200, 9999999999999
Reale	float	Numero a virgola mobile	3.14, 1.23e-10, 4.0E210
Booleano	bool	Per valori veri o falsi	True, False
Stringhe	str	Usata per rappresentare testo	'', 'stefano', "l'acqua"
Liste	list	Una sequenza mutabile di oggetti	[], [1, 2, 3], ['Hello', 'World']
Tuple	tuple	Una sequenza immutabile di oggetti	(), (1, 2, 3), ('Python', 3)
Insiemi	set	Un'insieme di oggetti unici	{1, 2, 3}, {'World', 'Hello'}
Dizionari	dict	Una struttura che associa chiavi a valori	{}, {'nome': 'Ezio', 'cognome': 'Melotti'}

Stringhe - indexing

- In Python, è possibile accedere agli elementi di una sequenza, usando la sintassi nomeSequenza[indice]. Questo restituirà l'elemento in posizione indice (il primo elemento ha sempre indice 0!)
- È inoltre possibile specificare indici negativi che partono dalla fine della sequenza (l'ultimo elemento ha indice -1, il penultimo -2, ecc.)

```
s = 'Python'
s[0]  # elemento in posizione 0 (il primo) 'P'
s[-1]  # elemento in posizione -1 (l'ultimo) 'n'
s[4]  # elemento in posizione 5, ovvero quello con indice 4 'o'
```

Stringhe - slicing

- La sintassi sequenza[inizio:fine] ci permette di ottenere una nuova sequenza dello stesso tipo che include tutti gli elementi partendo dall'indice inizio (incluso) all'indice fine (escluso)

```
a[start:stop]  # elementi da start sino a stop-1
a[start:]      # elementi da start sino alla fine
a[:stop]       # elementi dall'inizio sino a stop-1
a[:]           # una copia dell'intero array
a[start:stop:step] # da start sino stop-1, ogni step elementi
```

Stringhe - slicing

- start e stop negativi

  a[-1] #l'ultimo elemento

  a[-2:] # gli ultimi due elementi

  a[:-2] # tutti tranne gli ultimi due elementi

- step negativo

  a[::-1] # tutti gli elementi, al contrario (reversed)

  a[1::-1] # I primi due elementi, reversed

  a[:-3:-1] # gli ultimi due elementi, reversed

  a[-3::-1] # tutti gli elementi esclusi gli ultimi due, reversed

Stringhe - slicing

- Esempi:

```
s = ‘Python’
s[0:2]  # sottostringa con elementi da 0 (incluso) a 2 (escluso) 'Py’
s[:2]   # dall'inizio all'elemento con indice 2 (escluso) 'Py’
s[3:5]  # dall'elemento con indice 3 (incluso) a 5(escluso) 'ho’
s[4:]   # dall'elemento con indice 4 (incluso) alla fine 'on’
s[-2:]  # dall'elemento con indice -2 (incluso) alla fine 'on'
```

Stringhe - sottostringhe

- Gli operatori **in** e **not in** possono essere usati per verificare se un elemento fa parte di una sequenza o no

```
'P' in s  # il carattere 'P' è contenuto nella stringa s, quindi torna True
'x' in s  # il carattere 'x' non è in s, quindi ritorna False
'x' not in s  # "not in" esegue l'operazione inversa, quindi torna True
'Py' in s  # la sottostringa 'Py' è contenuto nella stringa s, quindi True
'py' in s  # il controllo è case-sensitive, quindi torna False
```

Stringhe – metodi di classe

- CONCATENAZIONE: +
- RIPETIZIONE: *

  'Ba' + 'na' * 2

  'Banana’
- LUNGHEZZA: len()

  len(‘Banana’)

  6

Stringhe – metodi di classe

- str.format() è un metodo che permette di inserire valori variabili in una stringa

```
raggio = 8.4
area = 3.14 * raggio**2
circ = 2 * 3.14 * raggio
s = "L'area e’ {}, la circonferenza e’ {}."
s.format(area, circ)
"L'area e’ 221.5584, la circonferenza e’ 52.752."
```

Stringhe – metodi di classe

- stringa.capitalized(): converte il 1° carattere della stringa in lettera maiuscola
- stringa.count(): restituisce il numero di volte in cui una sottostringa è presente nella stringa considerata
- stringa.endswith(): verifica se la stringa termina col carattere inserito. Restituisce un booleano
- stringa.index(): restituisce l'indice del carattere inserito
- stringa.isalpha(): non prende nessun argomento. Verifica se tutti i caratteri della stringa sono lettere dell'alfabeto

Stringhe – metodi di classe

- stringa.isdigit(): verifica se tutti i caratteri di una stringa sono numeri
- stringa.islower(): verifica se tutte le lettere della stringa sono minuscole
- stringa.lower(): trasforma tutte le lettere maiuscole in minuscole
- Analogamente stringa.isupper() e stringa.upper(), in maiuscole
- stringa.join(): restituisce una stringa con gli elementi di un iterabile. Se l'iterabile non dovesse contenere stringhe —> TypeError.!

Stringhe – metodi di classe

- stringa.strip(): restituisce la stessa stringa senza gli elementi inseriti
- stringa.split(): divide la stringa nel carattere inserito e restituisce una lista
- stringa.startswith(): verifica se la stringa inizia col carattere inserito
- Attenzione: le stringhe sono degli oggetti immutabili, non è possibile effettuare il seguente assegnamento: stringa[0] = ’n’.

Esercizi

Esercizio 1

Scrivere una funzione che accetti come parametro una stringa e che verifichi se nella stringa sono presenti come caratteri SOLO le cifre da 0 a 9.

Restituisce True o False.

Provarla con "oojo 1p0 ", "ciao ", "9087687"

Esercizio 2

Scrivere una funzione che accetti come parametro una stringa e che verifichi se nella stringa è presente almeno un carattere di punteggiatura.

Restituisce true o false.

Esercizio 3

Scrivere una funzione che accetta una stringa e che, se esiste un punto '.' nella stringa, ne restituisce la prima parte, cioe' quella che precede il punto.

Esempio: "ciao .a tutti. ecco" -> "ciao "

Esercizio 4

Scrivere una funzione che accetta una stringa e che, se esiste un punto '.' nella stringa, ne restituisce l’ultima parte, cioe' quella che segue il punto.

Esempio: "ciao .a tutti. ecco" -> "ecco "

Esercizio 5

Scrivere una funzione che chieda all’utente di inserire una parola e che sia in grado di capire se la parola inserita è un palindromo o no.

Torna True o False.

Esempi: radar, ingegni, osso.

Esercizio 6

Creare un programma che chieda all'utente di inserire 2 stringhe.

Il programma deve:

- creare una nuova stringa uguale alla somma delle lettere finali delle due stringhe
- restituire le due stringhe al contrario
  - 1. usando lo slicing
  - 2. usando il metodo reversed()
  - 3. definendo una funzione ricorsiva

Esercizio 7

Scrivere il codice di una funzione che accetti una stringa e un carattere e restituisca la posizione del carattere all’interno della stringa, -1 se non c’è.

Esercizio 8

a) Scrivere il codice di una funzione che, data una stringa e un carattere, verifica se il carattere compare doppio (consecutivamente) nella stringa.

b) Scrivere una seconda funzione che data una stringa verifica se al suo interno ci siano doppie (2 caratteri uguali consecutivi), se sì li stampa.

Esercizio 9

Scrivere una funzione che chieda all’utente di inserire una frase di almeno 20 lettere e che ripeta la richiesta fino a che non sia soddisfatta.

Esercizio 10

Scrivere una funzione che accetti in ingresso una stringa e che stampi il numero di lettere minuscole e maiuscole presenti nella stringa.

Si accettano anche stringhe contenenti sia lettere che numeri/caratteri (esempio “ciao45”), mentre non si accettano stringhe contenenti solo caratteri (esempio “!##+”), in tal caso la funzione deve stampare il messaggio: “La stringa di input non contiene lettere”.

Liste

Python – Tipi di dato

Tipo di dato	Nome	Descrizione	Esempi
Intero	int	Intero di dimensione arbitraria	-42, 0, 1200, 9999999999999
Reale	float	Numero a virgola mobile	3.14, 1.23e-10, 4.0E210
Booleano	bool	Per valori veri o falsi	True, False
Stringhe	str	Usata per rappresentare testo	'', 'stefano', "l'acqua"
Liste	list	Una sequenza mutabile di oggetti	[], [1, 2, 3], ['Hello', 'World']
Tuple	tuple	Una sequenza immutabile di oggetti	(), (1, 2, 3), ('Python', 3)
Insiemi	set	Un'insieme di oggetti unici	{1, 2, 3}, {'World', 'Hello'}
Dizionari	dict	Una struttura che associa chiavi a valori	{}, {'nome': 'Ezio', 'cognome': 'Melotti'}

Liste

- Le liste vengono definite elencando tra parentesi quadre [ ] una serie di oggetti separati da virgole
- A differenza di tuple e stringhe che sono immutabili, le liste possono essere mutate

```
nums = [0, 1, 2, 3]  # nuova lista di 4 elementi
type(nums)  # verifichiamo che il tipo sia "list"
empty = []  # nuova lista vuota
one = ['Python']  # nuova lista con un elemento
letters = ['a', 'b', 'c', 'd', 'e']
```

Liste – metodi utili

```
letters = ['a', 'b', 'c']
letters.append('d')  # aggiunge 'd' alla fine  ['a', 'b', 'c', 'd']
letters.extend(['e', 'f'])  # aggiunge 'e' e 'f' alla fine  ['a', 'b', 'c', 'd', 'e', 'f']
letters.append(['e', 'f'])  # aggiunge la lista come elemento alla fine
                                                ['a', 'b', 'c', 'd', 'e', 'f', ['e', 'f']]
letters.pop()  # rimuove e ritorna l'ultimo elemento (la lista) ['e', 'f']
letters.pop(0)  # rimuove e ritorna l'elemento in posizione 0 'a'
letters.remove('d')  # rimuove l'elemento 'd'
```

Liste – metodi utili

```
letters.reverse()  # inverte l'ordine "sul posto" e non ritorna niente
letters  # gli elementi sono ora in ordine inverso ['e', 'c', 'b']
letters[1] = 'x'  # sostituisce l'elemento in posizione 1 ('c') con 'x'
letters       # ['e', 'x', 'b']
del letters[1]  # rimuove l'elemento in posizione 1 ('x')
letters       # ['e', 'b']
letters.clear()  # rimuove tutti gli elementi rimasti
letters       # []
```

Esercizi

Esercizio 1

Scrivere una funzione che prenda come argomento una lista di numeri.

Tale funzione deve restituire la media e la deviazione standard dei numeri presenti all’interno della lista.

Esercizio 2

Scrivere una funzione che chieda all’utente di inserire una serie di elementi (numeri, stringhe). Questi elementi dovranno essere inseriti in una lista. La funzione deve restituire una nuova lista contenente le coppie (index,item) della lista precedente.

Esercizio 3

Creare la propria funzione sum di somma.

Prende in ingresso una lista e torna la somma dei suoi elementi.

Esempio:

sum([1, 2, 3, 4]) " 10

Esercizio 4

Scrivere una funzione che chiede all’utente di inserire dei numeri interi separati da uno spazio. La funzione stamperà le tabelline dei numeri inseriti.

Esercizio 5

Scrivere una funzione che restituisca in output i numeri della sequenza di Fibonacci, entro una soglia specifica impostata dall’utente.

NOTA: Nella serie di Fibonacci, ciascun numero della serie è la somma dei due numeri che lo precedono

Esercizio 6

Creare la propria funzione sort che ordini gli elementi della lista in ordine crescente.

Esempio: [4, 5, 2, 7, 8] -> [2, 4, 5, 7, 8]

Esercizio 7

Scrivere un programma che chieda all’utente di inserire dei voti, SI/NO, fino a che non si inserisce zero e conti quanti SI e quanti NO sono stati inseriti e dichiari chi ha vinto o se sono pari.

Reminder

Try e Except

- Per la gestione degli errori
- Il blocco ‘try’ consente di testare un blocco di codice per gli errori
- Il blocco ‘except’ consente di gestire l'errore
- Il blocco ‘finally’ ti consente di eseguire il codice, indipendentemente dal risultato dei blocchi try-except
- Il blocco ‘else’ si puo’ usare per eseguire del codice se non sono stati generati errori
- E’ possibile avere diversi except, seguiti dal nome dell’errore

Try e Except: esempio

```
try:
    print(x)
except NameError:
    print("Variable x is not defined")
except:
    print("Something else went wrong")
else:
  print("Nothing went wrong")
finally:
  print("The 'try except' is finished")
```